Chunk 1 Chunk 2

A Complete Guide to

RAG Chunking

Professional Notes

1. Why Chunking Exists in RAG

RAG (Retrieval-Augmented Generation) works in three stages:

1. **Chunk documents**
2. **Embed chunks**
3. **Retrieve top-k chunks → Send to LLM**

Chunking is the foundation of the entire system.

If chunking is wrong:

- Retrieval fails
- Hallucinations increase
- Context gets fragmented
- Token costs explode

You are not chunking text.

You are chunking **knowledge units**.

---

2. What a Chunk Really Is

A chunk is the smallest retrievable unit of meaning.

Bad chunk:

```arduino
"The transformer architecture was introduced in 2017. Attention allows the model to..."
```

(cut mid-idea)

Good chunk:

```arduino
"What is the Transformer architecture? It was introduced in 2017 by Vaswani et al. and us
```

(one complete thought)

3. Chunking Controls the RAG Failure Modes

Failure	Root Cause
LLM makes things up	Missing chunk
Wrong answer	Chunk too small
Vague answer	Chunk too large
High cost	Over-long chunks
Low recall	Chunk boundaries break meaning

Chunking is not preprocessing.

It is **retrieval architecture**.

---

4. Core Chunking Parameters

4.1 Chunk Size

Measured in tokens or characters.

Use Case	Recommended Size
FAQs	200–400 tokens
Documentation	400–800 tokens
Legal / Contracts	800–1200 tokens
Code	200–500 tokens

Why?

- Embeddings lose precision after ~800 tokens
- LLM context gets polluted with noise

4.2 Overlap

Chunks must overlap so knowledge isn't cut.

Example with 20% overlap:

```yaml
Chunk 1: tokens 0–400
Chunk 2: tokens 320–720
Chunk 3: tokens 640–1040
```

Overlap preserves:

- Definitions
- Cross-sentence logic
- References

Typical overlap: **10–25%**

---

5. The 5 Real Chunking Strategies

Let's break down what actually works in production.

5.1 Fixed-Size Chunking

Split by token count.

```sql
Every 500 tokens → new chunk
```

Pros

- Fast
- Simple
- Works for uniform text

Cons

- Breaks sentences
- Breaks tables
- Breaks meaning

5.2 Sentence-Based Chunking

Split by sentence boundaries.

```pgsql
Group sentences until token limit is hit
```

Much better than fixed size.

Pros

- Preserves grammar
- Less semantic damage

Cons

- Still breaks topic boundaries

5.3 Semantic Chunking

This is where real RAG begins.

Chunks are split when topic changes.

You use:

- Sentence embeddings
- Cosine similarity
- Break where similarity drops

Example:

```java
If similarity(sentence_i, sentence_i+1) < threshold → new chunk
```

This produces:

- Concept-aligned chunks
- Self-contained knowledge blocks

This is the default for high-quality RAG systems.

5.4 Document-Structure Chunking

Split by:

- Headers
- Sections
- Paragraphs
- Bullet groups

Example:

```shell
## Installation
→ One chunk

## Configuration
→ One chunk

## API Reference
→ Sub-chunks per endpoint
```

This is perfect for:

- Docs
- Wikis
- Policies
- Research papers

Always preserve headings inside chunks.

5.5 Hybrid Chunking (Production Standard)

Best practice:

1. Split by document structure
2. Inside each section, apply semantic chunking
3. Apply size limits + overlap

This creates:

- Logically coherent chunks
- Embedding-friendly size
- Retrieval-optimized knowledge blocks

6. Chunk Metadata Is as Important as Text

Every chunk should store:

```json
{
  "chunk_id": "doc1_sec3_p2",
  "document": "API_Guide.pdf",
  "section": "Authentication",
  "page": 12,
  "tokens": 480
}
```

Why?

Because retrieval without metadata is blind.

Metadata enables:

- Filtering
- Source citation
- Page-level grounding
- Reranking

---

7. What Happens After Chunking

The real pipeline:

```scss
Raw docs
→ Chunking
→ Embedding each chunk
→ Vector DB
→ Query embedding
→ Similarity search
→ Top-k chunks
→ Reranker (optional)
→ LLM
```

7. What Happens After Chunking

The real pipeline:

```scss
Raw docs
→ Chunking
→ Embedding each chunk
→ Vector DB
→ Query embedding
→ Similarity search
→ Top-k chunks
→ Reranker (optional)
→ LLM
```

Bad chunking = garbage embeddings.

---

8. Chunk Size vs Retrieval Accuracy

Here's the tradeoff:

Chunk Size	Retrieval	LLM Quality
Too small	High recall, low precision	Fragmented answers
Too large	Low recall	Irrelevant context
Just right	High recall + precision	Clean answers

The sweet spot is:

300–800 tokens with semantic boundaries

9. How Chunking Impacts Hallucinations

LLMs hallucinate when:

- Required fact is not in retrieved chunks
- Fact is split across chunks
- Chunk is too noisy

Good chunking reduces hallucinations more than:

- Prompt engineering
- Temperature tuning
- Model size

10. Special Chunking Cases

10.1 Tables

Do NOT split rows across chunks.

Store tables as:

- CSV-like text
- Or one chunk per table

10.2 Code

Chunk by:

- Function
- Class
- File

Never chunk by token count for code.

10.3 PDFs

Chunk by:

- Page → Section → Paragraph

Preserve page numbers in metadata.

---

11. How to Know If Your Chunking Is Bad

Ask your RAG system:

- What is X?
- Where is X defined?
- How does X work?

If answers are:

- Vague
- Half-correct
- Missing details